



Objectif : acquérir les savoir-faire élémentaire des techniques de programmation en C sous Linux.

Remarque : la connaissance du C n'est pas indispensable, la lecture des codes C du TP étant relativement intuitive. Le cours/TP permet d'appréhender la structure d'une chaîne de compilation et les options élémentaires du compilateur et du linker.

1 Compilation C sous LINUX

Le C possède un jeu d'instructions enrichis par de nombreuses librairies. Le compilateur GNU C un compilateur libre de droit et intégré dans toutes les distributions Linux. C'est avec celui-ci que nous allons travailler.

Généralement le développement logiciel quelque soit les langage s'effectue à travers un environnement de développement intégré (EDI ou IDE en anglais). Les plus célèbres sont CODEBLOCKS, Qt, Netbeans ou Eclipse ou pour le C/C++.

Ces logiciels sont relativement consommateurs de ressources système et nécessitent une bonne bande passante pour le développement à distance.

Pour le développement à distance on préfère généralement travailler en console à travers une liaison sécurisée par SSH (Secure Shell).

Ce TP propose une découverte de ce mode de développement.

Construction d'un programme C

L'écriture d'un programme C donne lieu à la création d'un ou plusieurs fichiers sources (extension .c). La création de fichiers header personnels (extension .h) permet de définir les fonctions créées par l'utilisateur. L'appel aux compilateur GNU C gcc peut se faire en ligne de commande

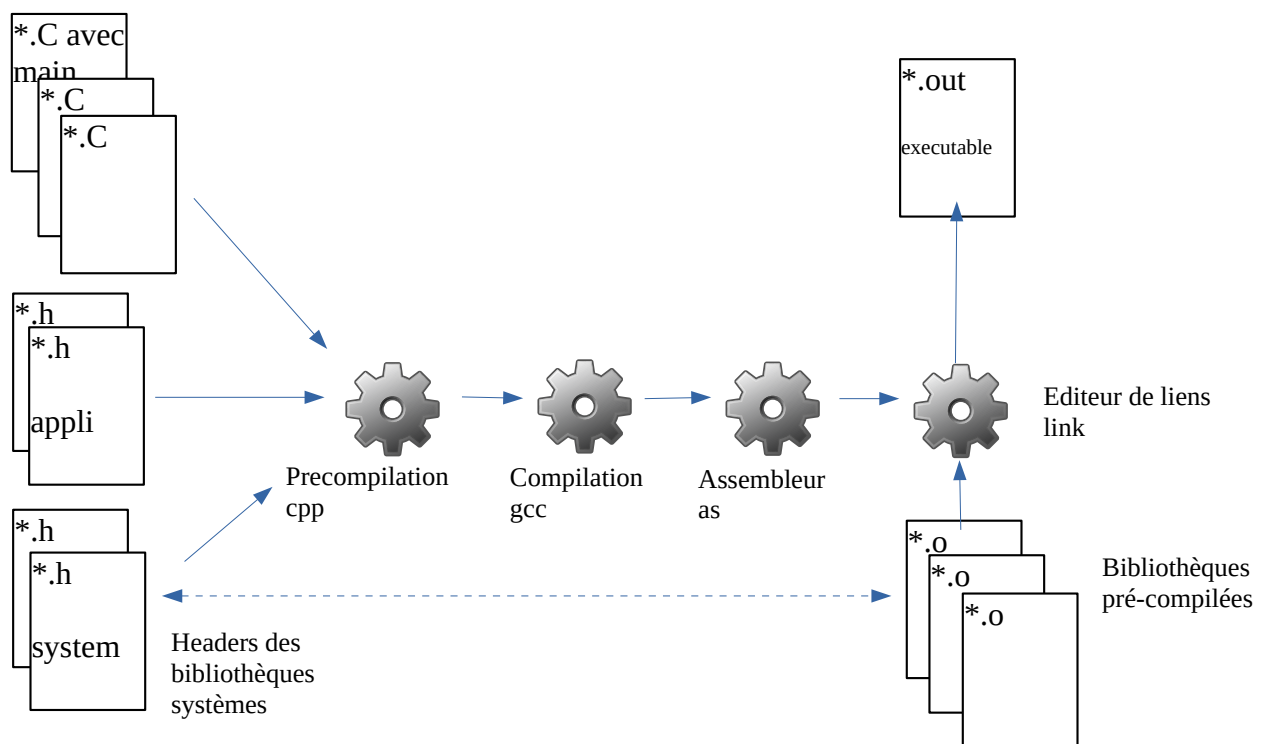
```
gcc source.c
```

Cette ligne de commande crée un fichier binaire exécutable a.out.

On peut nommer le fichier binaire avec l'option -o du compilateur :

```
gcc source.c -o binaire
```

La création d'un fichier exécutable à partir de fichiers sources en C fait appel à plusieurs programmes :





Le pré compilateur (cpp) supprime les commentaires et interprète les directives de compilations qui se trouvent dans les fichiers header ou sources.

gcc génère un fichier texte en langage assembleur.

L'assembleur (as) crée un fichier objet correspondant au fichier source qui sera ensuite lié aux bibliothèques système pré-compilées pour produire l'exécutable

Les options du compilateur

Nous venons de voir une première option (-o) du compilateur. Il en existe de nombreuses (voir le manuel en ligne).

-v Visualise toutes les actions effectuées lors d'une compilation.

-c Supprime l'édition de lien. Seul le(s) fichier(s) .o est (sont) généré(s) pour réaliser des bibliothèques pré-compilées par exemple..

-o Spécifie le nom de l'exécutable à générer.

-S Ne génère pas le fichier exécutable et crée un fichier .s contenant le programme en assembleur.

-L Indique un chemin de recherche supplémentaire à l'éditeur de liens pour d'autres bibliothèques.

-I Indique une nouvelle bibliothèque à charger.

Exercice

A l'aide de l'éditeur Nano, créez le fichier source hello.c qui contient les instructions C suivantes :

```
#include <stdio.h>
int main(void)
{
    printf("bonjour\n");
    return(0);
}
```

Compilez-le pour créer l'exécutable : gcc hello.c

Lancer exécutable : ./a.out (./ indique à Linux d'essayer d'exécuter le fichier)

gcc hello.c -o monprog.out remplacera a.out par monprog.out

Recompilez ce programme en spécifiant l'option -v et commentez ce qui s'affiche.

Trouvez l'option pour ne générer que le fichier assembleur puis objet de votre source.

Éditer le fichier assembleur avec Nano, vous constaterez que ce langage n'est pas d'un accès facile.

Éditer maintenant le fichier exécutable, vous y trouverez le code machine (totalement illisible) ainsi que des informations sur les bibliothèques utilisées ainsi que sur l'OS

```
.file "hello.c"
.text
.section .rodata
.LC0:
.string "Bonjour\n"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
leaq .LC0(%rip), %rdi
movl $0, %eax
call printf@PLT
movl $0, %eax
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE0:
.size main, .-main
.ident "GCC: (Ubuntu 7.3.0-16ubuntu3) 7.3.0"
.section .note.GNU-stack,"",@progbits
```



Les erreurs :

dans hello.c changer printf en print, recompiler

Lisez en détail les informations d'erreur fournies par le compilateur.

```
christian@christian-mint-VB: ~/C
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
christian@christian-mint-VB:~/C$ gcc hello.c
hello.c: In function 'main':
hello.c:5:2: warning: implicit declaration of function 'print'; did you mean 'printf'?
[-Wimplicit-function-declaration]
  print("Bonjour à tous\n\r");
  ^~~~~
  printf
/tmp/ccXIW8CP.o : Dans la fonction « main » :
hello.c:(.text+0x11) : référence indéfinie vers « print »
collect2: error: ld returned 1 exit status
christian@christian-mint-VB:~/C$
```

- gcc indique ici que la fonction print n'a pas été déclarée (elle n'existe pas) et propose d'utiliser printf.
- Le symbole ^ indique l'emplacement de l'erreur
- Le message d'erreur est très explicite : référence indéfinie vers « print ». Le linker ne sait pas où est la fonction print.

Corriger l'erreur et essayer une erreur de syntaxe en retirant un ;

2 Construction de bibliothèque (librairie en anglais)

Il est intéressant de se construire des bibliothèques contenant les fonctions les plus fréquemment utilisées plutôt que de les réécrire à chaque projet. Il suffit ensuite d'indiquer vos librairies au moment de la compilation. Pour cela, les options -L et -l permettent respectivement d'inclure un nouveau chemin de recherche pour l'éditeur de lien et d'indiquer le nom de librairie.

Vous trouverez ci-dessous 2 fonctions qui affichent "Bonjour" et "Au revoir" autant de fois que le nombre passé en argument :

```
void bonjour(int nbre)
{
    int i;
    for(i=0;i<nbre;i++) printf("Bonjour ...\n");
    return;
}

void revoir(int nbre)
{
    int i;
    for(i=0;i<nbre;i++) printf("Au revoir ...\n");
    return;
}
```

Placez ces fonctions dans des fichiers nommés bonjour.c et revoir.c. Compilez ces fichiers afin de générer les fichiers objet :

```
gcc -c bonjour.c revoir.c    Des avertissements apparaissent (warning)...
```



La commande ls montrera que le compilateur a créé deux fichiers objets en code machine `bonjour.o` et `revoir.o`.

Il est maintenant possible de lier ces fichiers objet lors d'une compilation.

Il suffira de les déclarer dans un fichier header, la compilation ne sera plus nécessaire d'où un gain de temps.

libsalut.h

```
#ifndef libslt
#define libslt
void bonjour(int nbre) ;
void revoir(int nbre) ;
#endif
```

Tapez le programme suivant qui appelle des fonctions de `bonjour.c` et `revoir.c` dans le fichier `exemple2.c` :

```
#include <stdio.h>
#include "libsalut.h"

int main(void)
{
    int a=0;
    printf("Donnez un nombre entre 1 et 5 compris: ");
    scanf("%d",&a);
    printf("\n");
    bonjour(a);
    revoir(a);
    return(0);
}
```

La compilation peut se faire par

```
gcc exemple2.c bonjour.o revoir.o
```

Ici seul `exemple2.c` est compilé pour générer `exemple2.o` qui sera lié à `salut.o` et `revoir.o`.

Les fichiers objet `*.o` peuvent être regroupés dans une bibliothèque.

Créez la librairie `libsalut.a` (les noms des bibliothèques commencent toujours par `lib`) à l'aide de la commande `ar` qui crée un fichier archive :

```
ar r libsalut.a bonjour.o revoir.o
```

Vous pouvez vérifier le contenu de votre librairie en tapant :

```
ar t libsalut.a
```

Utilisation de la librairie

A partir de cette étape, vous pouvez déjà utiliser ces fonctions en respectant quelques conditions.

La première de celles-ci est de placer `libsalut.a` dans le répertoire où vous effectuez votre compilation. La seconde est de compiler votre programme en spécifiant le nom de la librairie.

Compilez ce programme en exécutant la ligne suivante :

```
gcc -o exemple exemple2.c libsalut.a
```

Vous pouvez également placer toutes vos librairies dans un de vos répertoires (par exemple, « librairies » que vous créerez par `mkdir librairies`).



Vous lancerez alors votre compilation de la manière suivante :

```
gcc -o exemple exemple2.c -L librairies -l salut
```

-L indique au linker un nouveau chemin pour les bibliothèques

-l indique le nom de la bibliothèque, remarquez que le nom de la bibliothèque libsalut.a devient salut dans la ligne de commande.

3 Compilation séparée

Lorsque vous réalisez de gros projets, vous devez découper ceux-ci en plusieurs programmes sources. L'avantage est un gain de temps au moment de la compilation. Si une application est constituée des 3 fichiers sources source1.c, source2.c et source3.c la compilation d'un exécutable nommé exec est effectuée par la ligne suivante :

```
gcc -o exec source1.c source2.c source3.c
```

L'option -c du compilateur permet de générer des fichiers objets sans effectuer une édition de lien. Si vous modifiez un des fichiers source (par exemple, source3.c), nous utilisons cette option pour ne compiler que le fichier modifié :

```
gcc -c source3.c  
gcc -o exec source3.o source1.o source2.o
```

Cette technique devient indispensable lors de la création de gros logiciels qui peuvent nécessiter des temps de compilation de plusieurs minutes, ou dans le cas d'un travail collaboratif.

4 Exercice

On réalise un programme demandant deux nombres et affichant la somme et la différence de ces nombres.

```
Fichier  Édition  Affichage  Rechercher  Terminal  Aide  
christian@christian-mint-VB:~/C/exo1$ ./a.out  
entrer le premier nombre : 23  
entrer le deuxième nombre : 65  
  
23 + 65 = 88  
  
23 - 65 = -42  
christian@christian-mint-VB:~/C/exo1$
```

Le programme source est découpé en trois fichiers

**Fichier main.c**

```
#include <stdio.h>
#include "calcul.h"

int main(void)
{
    int a,b,s,d;
    printf("entrer le premier nombre : ");
    scanf("%d",&a);
    printf("entrer le deuxième nombre : ");
    scanf("%d",&b);
    s=somme(a,b);
    d=diff(a,b);
    printf("\n%d + %d = %d\n",a,b,s);
    printf("\n%d - %d = %d\n",a,b,d);
    return(0);
}
```

Fichier somme.c

```
int somme(int x, int y)
{
    return (x+y);
}
```

Fichier diff.c

```
int diff(int x, int y)
{
    return(x-y);
}
```

1. Créer le fichier **calcul.h** contenant les prototypes des fonctions « somme » et « diff ».
2. Compiler les fichiers main.c somme.c diff.c de manière à obtenir l'exécutable « calc » (tester)
3. Compiler somme.c et diff.c de manière à créer les fichiers objets somme.o et diff.o
4. Re-crée calc en compilant et liant main.c somme.o et diff.o

On souhaite modifier le programme de manière à retourner toujours un nombre positif pour la différence en ajoutant à diff.c

```
if (x>y) return(x-y);
else return(y-x);
```

5. Modifier uniquement diff.c, créer uniquement un nouvel objet diff.o et recompiler l'ensemble comme précédemment. Tester
6. Créer dans votre répertoire de travail une bibliothèque malib/libcalcul.a contenant les fichiers objets somme.o et diff.o (seul main.c et calcul.h resteront dans le dossier de travail)
7. Donner la ligne de commande permettant de compiler main.c avec cette bibliothèque. Tester